

Cloud Computing Tools

Workshop I: *Pipeline de datos automatizado, calidad de datos y arquitectura Medallion sobre AWS S3*

Juan Pablo Jiménez — Maestría en Analítica Aplicada

Harrison Alonso — Maestría en Analítica Aplicada

Juan Fernando Serrano — Maestría en Analítica Aplicada

Universidad de La Sabana

Maestría en Analítica Aplicada

Prof.: Hugo Franco, Ph.D.

21 de agosto de 2026

Código fuente: <https://github.com/juanferrrr/Deliverable-1-Grupo-Alonso-Jimenez-Serrano>

1. Orquestación y manejo de errores

1.1. Descripción del problema

Hasta este punto la solución consistía en tres scripts de Python independientes —`data_ingestion.py`, `data_loading.py` y `data_analysis.py`— que ejecutábamos a mano, uno tras otro, en un orden fijo. Ese arreglo es suficiente para explorar una API en un ejercicio de clase, pero falla como artefacto de ingeniería por tres razones. Primero, el orden de ejecución vive en la memoria del operador y no en el código, de modo que nada impide correr el análisis antes que la ingesta. Segundo, una falla en un paso intermedio deja el Data Lake en un estado inconsistente: puede quedar cargado un archivo crudo mientras el archivo transformado que debería acompañarlo nunca se materializa, y nada en el sistema registra esa inconsistencia. Tercero, las fallas se manifiestan como *tracebacks* de Python, que indican qué se rompió pero no qué debe hacerse al respecto.

Experimentamos el tercer problema de forma directa durante el desarrollo. Las credenciales temporales que emite el AWS Academy Learner Lab expiraron a mitad de sesión y el script de análisis terminó con el mensaje “*An error occurred (ExpiredToken) when calling the GetObject operation: The provided token has expired*”. El mensaje es técnicamente correcto y operativamente inútil: no dice que las credenciales del Learner Lab tienen una vigencia de cuatro horas, ni que deben obtenerse unas nuevas desde el panel “AWS Details”, ni cuáles variables del archivo `.env` hay que reemplazar. Diagnosticarlo nos costó más tiempo del que la falla merecía.

La tarea de este punto fue, entonces, consolidar los tres scripts en un único orquestador construido con funciones modulares, y garantizar que cada modo de falla previsible produzca una detención controlada acompañada de un mensaje que indique la acción correctiva.

1.2. Método de solución

Datos. La fuente es la Foursquare Places API (endpoint de búsqueda de lugares), consultada para establecimientos de tipo *restaurants* alrededor de las coordenadas (4,85876; -74,05866), correspondientes al casco urbano de Chía, Cundinamarca. El endpoint devuelve un documento JSON cuyo arreglo **results** contiene un objeto por establecimiento. La Tabla 1 lista las variables que conservamos y el tipo con el que se modela cada una en el DataFrame de Pandas.

Variable	Tipo	Descripción
name	object (str)	Nombre comercial del establecimiento.
address	object (str)	Dirección; usa formatted_address como respaldo.
category_name	object (str)	Nombre de la categoría principal, aplanado del objeto anidado.
latitude	float64	Latitud en grados decimales (WGS 84).
longitude	float64	Longitud en grados decimales (WGS 84).
distance	int64	Distancia en metros desde las coordenadas de referencia.

Tabla 1: Esquema del conjunto de datos ingestado y tipo asignado a cada variable.

Fuente: elaboración propia.

Dos aspectos de la ingesta merecen comentario. La API devuelve las categorías como una lista de diccionarios anidados de la forma [{"fsq_category_id": "...", "name": "Steakhouse", ...}]. Persistir esa estructura en un archivo CSV la serializa como la representación textual del diccionario, lo cual la vuelve inutilizable para agrupar: cualquier agregación posterior agruparía por la cadena completa, incluidos identificadores y URL de iconos. Por eso aplanamos la estructura en el momento de la ingesta, extrayendo únicamente el nombre de la categoría principal. De forma análoga, el campo **address** está ausente con frecuencia —en nuestra primera ingesta 24 de 50 registros no tenían dirección— así que recurrimos a **formatted_address** cuando el campo primario es nulo.

La petición se emite con **limit** = 50, que es el máximo que acepta el endpoint; el valor por defecto es 10 (Foursquare, 2026). Recuperar más de cincuenta establecimientos exige paginación basada en cursor, que no implementamos. Las consecuencias de esa decisión se examinan en la discusión.

Orquestación. Empleamos Prefect 3 como marco de orquestación. Cada etapa del pipeline es una función ordinaria de Python decorada con **@task**, y el orquestador es una función decorada con **@flow** que las invoca en secuencia. Prefect aporta tres cosas que una simple sucesión de llamadas no ofrece: seguimiento de estado independiente por tarea (cada una termina en un estado *Completed* o *Failed* que queda registrado), registro estructurado que atribuye cada mensaje a la tarea que lo emitió, y políticas declarativas de reintento.

Los reintentos se aplican únicamente a la tarea de ingesta (**retries** = 2, **retry_delay_seconds** = 5). El razonamiento es que una falla de red contra una API externa suele ser transitoria, mientras que los demás modos de falla de este pipeline no lo son: una llave de API ausente, una credencial vencida o un archivo local inexistente fallarán de forma idéntica en el segundo intento, de modo que reintentar solo retrasaría el diagnóstico.

Modelo de errores. Definimos una única excepción propia, `PipelineError`, que representa una falla prevista. Cada tarea convierte las excepciones específicas de librería que pueda encontrar — `requests.exceptions.Timeout`, `requests.exceptions.HTTPError`, `botocore.exceptions.ClientError`, `OSError`, `duckdb.Error`— en una `PipelineError` cuyo mensaje enuncia la acción correctiva. El *flow* envuelve toda la secuencia en un `try/except` que captura `PipelineError`, la registra y devuelve `False` sin propagar el *traceback*; esta es la degradación elegante que exige el enunciado. Una segunda cláusula `except Exception`, más amplia, actúa como red de seguridad, de modo que un defecto imprevisto se reporta con su tipo y mensaje en lugar de tumbar el proceso.

Una función auxiliar, `translate_s3_error()`, inspecciona el código de error que AWS devuelve dentro de la excepción (`error.response[.Error][Code]`) y lo traduce a una instrucción de remediación. Los códigos `ExpiredToken`, `ExpiredTokenException`, `InvalidToken` y `RequestExpired` se agrupan, porque AWS reporta la misma condición subyacente con nombres distintos según la operación; `NoSuchBucket`, `NoSuchKey` y `AccessDenied` reciben cada uno su propio mensaje. La consulta se hace con llamadas encadenadas a `.get()` en lugar de indexación directa, para que una estructura de respuesta inesperada no pueda hacer fallar al propio manejador de errores.

Algoritmo 1. Orquestador (pseudocódigo)

Funcion `medallion_pipeline():` booleano

```

intentar
    df_raw    <- ingest_places()           # API Foursquare, reintentos = 2
    save_raw_csv(df_raw)                  # persistencia local
    upload_to_s3(LOCAL_RAW, RAW_KEY)      # raw_zone/

    df_lake   <- read_raw_from_s3()        # relectura desde el lake
    df_clean  <- validate_quality(df_lake) # aborta bajo el umbral
    write_optimized_parquet(df_clean)
    upload_to_s3(LOCAL_OPT, OPTIMIZED_KEY) # optimized_zone/

    build_consumption_layer()             # agregacion con DuckDB
    upload_to_s3(LOCAL_KPIS, CONSUMPTION_KEY) # consumption_zone/

    verify_zones()                        # list_objects_v2
    benchmark_columnar_scaling(df_clean)

excepto PipelineError como e
    log_error("PIPELINE DETENIDO: " + mensaje(e))
    retornar falso
excepto Exception como e
    log_error("ERROR INESPERADO (" + tipo(e) + "): " + mensaje(e))
    retornar falso

log_info("Pipeline completado con exito")
retornar verdadero

```

El método está implementado en el archivo adjunto `pipeline.py`, disponible en el repositorio referenciado en la portada. El modelo de tareas y flujos de Prefect sigue el patrón documentado por Prefect Technologies (2026).

1.3. Resultados

El pipeline se ejecuta con el comando `python pipeline.py` desde la raíz del proyecto. Requiere un archivo `.env` en el mismo directorio que contenga `FOURSQUARE_API_KEY`, `S3_BUCKET` y las tres variables de AWS que emite el Learner Lab (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`), junto con `AWS_DEFAULT_REGION`. No se necesitan más parámetros; las coordenadas de búsqueda, las rutas del bucket y el umbral de calidad están declarados como constantes en el encabezado del módulo.

La Figura 1 reproduce la salida de consola de la ejecución del 20 de agosto de 2026 a las 20:48. Las ocho tareas se completaron en aproximadamente 4,5 segundos de tiempo de reloj y todas terminaron en estado *Completed*. Prefect antepone a cada línea la tarea que la produjo, lo cual permite leer la traza como una narración del pipeline y no como un registro indiferenciado.

```
Flow run 'affable-yak' - Iniciando pipeline sobre el bucket s3://juanpablojimenez
Task run '1. Ingesta - Foursquare API' - Ingesta exitosa: 50 lugares recuperados.
Task run '2. Persistencia local del CSV crudo' - CSV crudo guardado (4,220 bytes)
Task run 'Carga a S3' - Subido a s3://juanpablojimenez/raw_zone/restaurants_raw.csv
Task run '3. Lectura desde la capa Raw (S3)' - 50 registros, 6 columnas.
Task run '4. Validacion de calidad' - Registros validos: 50 (100.0%)
Task run '4. Validacion de calidad' - Umbral de calidad superado. El pipeline continua.
Task run '5. Capa Optimized - Parquet' - Parquet generado (6,955 bytes)
Task run 'Carga a S3' - Subido a .../optimized_zone/restaurants_processed.parquet
Task run '6. Capa Consumption - DuckDB' - Capa de consumo generada: 26 categorias.
Task run 'Carga a S3' - Subido a .../consumption_zone/category_kpis.parquet
Task run '7. Verificacion FinOps' - Verificacion FinOps completada.
Task run '8. Benchmark de escalamiento columnar' - Benchmark completado.
Flow run 'affable-yak' - Pipeline completado con exito. Las tres zonas actualizadas.
Flow run 'affable-yak' - Finished in state Completed()
```

Figura 1: Salida de consola abreviada de una ejecución exitosa (20 de agosto de 2026, 20:48). Se suprimieron las marcas de tiempo y las líneas de estado de Prefect por legibilidad; el identificador de la corrida y las cifras reportadas no fueron modificados.

Fuente: elaboración propia.

La ruta de degradación elegante se ejercita con las mismas condiciones que la motivaron. Cuando las credenciales del Learner Lab expiran, el pipeline ya no termina con un *traceback* de `botocore`: el *flow* reporta “*PIPELINE DETENIDO: Las credenciales temporales de AWS Academy expiraron. Inicia una nueva sesión del Learner Lab (Start Lab), abre ‘AWS Details’ y actualiza las tres variables AWS_* en tu archivo .env*”, devuelve un código de salida distinto de cero y deja el Data Lake intacto desde el punto de falla en adelante.

2. Validación de calidad de datos

2.1. Descripción del problema

El enunciado del taller establece que la capa optimizada debe contener únicamente datos limpios y confiables. El requisito no es meramente estético: un registro inválido que llega a la capa optimizada se propaga a todos los consumidores aguas abajo, y como la capa de consumo está preagregada, un solo valor corrupto puede distorsionar una métrica que un tablero presenta luego como un hecho.

Los datos que ingestamos están particularmente expuestos a este problema. Foursquare se nutre en buena medida de aportes de usuarios, de modo que la completitud de sus registros varía: algunos establecimientos traen dirección completa y coordenadas precisas, mientras que otros fueron registrados con información parcial. Un registro sin coordenadas no sirve para un análisis geoespacial, y una distancia negativa es físicamente imposible e indicaría un defecto en la fuente o en nuestra propia transformación.

El problema tiene entonces dos niveles. En el nivel de registro debemos decidir qué filas individuales son admisibles. En el nivel de conjunto debemos decidir si el lote completo es lo bastante confiable como para publicarse: si una proporción alta de los registros falla la validación, la respuesta sensata no es publicar el remanente limpio sino sospechar que algo cambió en la fuente y detenerse.

2.2. Método de solución

Implementamos cuatro validaciones. Cada una produce una máscara booleana sobre el DataFrame; las máscaras se combinan con un AND lógico y se aplican una sola vez, de manera que un registro que incumple varios criterios se contabiliza bajo cada uno en el reporte pero se elimina una única vez.

ID	Validación	Regla y justificación
V1	Coordenadas válidas	<code>latitude</code> y <code>longitude</code> no nulas y dentro de $[-90, 90]$ y $[-180, 180]$ respectivamente. La verificación de rango detecta valores presentes pero imposibles, que una prueba de nulidad admitiría.
V2	Distancia no negativa	<code>distance</code> se convierte con <code>pd.to_numeric(errors=coerce)</code> y debe ser ≥ 0 . La coerción convierte un valor no numérico en NaN, que entonces falla la prueba, en lugar de lanzar una excepción.
V3	Nombre utilizable	<code>name</code> no nulo y no vacío tras eliminar espacios. Un establecimiento sin nombre no puede identificarse en ningún reporte.
V4	Sin duplicados	Se eliminan los registros duplicados en la terna (<code>name</code> , <code>latitude</code> , <code>longitude</code>), conservando la primera aparición. La paginación y las búsquedas solapadas pueden devolver dos veces el mismo lugar.

Tabla 2: Validaciones aplicadas al DataFrame antes de escribir la capa optimizada.

Fuente: elaboración propia.

Regla de negocio. Fijamos `MIN_QUALITY_RATIO = 0,80`, es decir, el umbral que sugiere el enunciado expresado como la proporción de registros que deben sobrevivir. Tras aplicar las cuatro

validaciones el pipeline calcula `ratio = sobrevivientes / total`; si esa proporción cae por debajo de 0,80 la tarea lanza una `PipelineError` y el *flow* se detiene antes de que ocurra cualquier carga a las zonas optimizada y de consumo. La zona cruda no se revierte, deliberadamente: el lote inválido queda disponible para inspección forense, que es precisamente para lo que existe la capa cruda.

Vale la pena explicitar una decisión de diseño. El umbral se evalúa después de la deduplicación, lo que significa que los duplicados cuentan en contra de la calidad del lote en lugar de tratarse como una limpieza inocua. Consideramos que es la lectura más estricta y defendible: una fuente que de pronto empieza a devolver el mismo establecimiento repetidamente ha cambiado de una manera que amerita atención humana.

Algoritmo 2. Validación de calidad de datos (pseudocódigo)

```
Funcion validate_quality(df: DataFrame): DataFrame

    total <- numero_de_filas(df)
    si total = 0 entonces lanzar PipelineError("conjunto de datos vacio")
    si columnas_requeridas no subconjunto_de columnas(df) entonces
        lanzar PipelineError("esquema inesperado en la capa cruda")

    lat <- a_numerico(df.latitude, coerce)
    lon <- a_numerico(df.longitude, coerce)
    dist <- a_numerico(df.distance, coerce)

    mascara_coords <- no_nulo(lat) Y no_nulo(lon)
                    Y lat en [-90, 90] Y lon en [-180, 180]
    mascara_dist <- no_nulo(dist) Y dist >= 0
    mascara_nombre <- no_nulo(df.name) Y recortar(df.name) <> ""

    limpio <- df[ mascara_coords Y mascara_dist Y mascara_nombre ]
    limpio <- quitar_duplicados(limpio, en = (name, latitude, longitude))

    ratio <- numero_de_filas(limpio) / total
    registrar_reporte_calidad(total, fallas_por_validacion, ratio)

    si ratio < MIN_QUALITY_RATIO entonces
        lanzar PipelineError("umbral de calidad no alcanzado: " + ratio)

    retornar limpio
```

2.3. Resultados

La tarea de validación emite un reporte a consola antes de decidir si continúa. La Tabla 3 transcribe el reporte producido por la ejecución del 20 de agosto de 2026.

Fuente: elaboración propia.

Verificación	Descartados	Proporción
Registros que entran a validación	50	100,0 %
V1 — coordenadas nulas o fuera de rango	0	0,0 %
V2 — distancia no numérica o negativa	0	0,0 %
V3 — nombre nulo o vacío	0	0,0 %
V4 — duplicados exactos	0	0,0 %
Registros que sobreviven la validación	50	100,0 %

Tabla 3: Reporte de calidad de datos para la ejecución del 20 de agosto de 2026.

Los cincuenta registros pasaron todas las validaciones, de modo que la proporción del 100 % superó holgadamente el umbral del 80 % y el pipeline continuó. Este resultado merece una interpretación honesta antes que presentarse como un éxito de la capa de validación. Lo explican dos factores. Primero, la función de ingesta ya normaliza los dos campos que antes resultaban problemáticos: aplanar el objeto anidado de categorías y sustituye `formatted_address` donde falta `address`, de modo que la incompletitud que habíamos observado en corridas anteriores ya no se manifiesta como un nulo en las columnas validadas. Segundo, los campos que examinan las validaciones —coordenadas y distancia— los calcula Foursquare y no los aportan los usuarios, y en consecuencia están bien formados para cualquier establecimiento indexado.

Las validaciones deben entenderse entonces como un contrato y no como una rutina de limpieza. Hoy no eliminan nada, pero garantizan que si la fuente se degrada, si la API cambia su esquema o si una modificación futura de la ingesta introduce un defecto, el pipeline lo detectará y se negará a publicar en lugar de propagarlo en silencio. Verificamos la ruta de aborto inyectando un lote sintético de quince registros en el que siete incumplían al menos una regla; la tarea reportó una proporción de supervivencia del 53,3 %, lanzó la `PipelineError` y no se escribió ningún objeto en las zonas optimizada ni de consumo.

3. Arquitectura Medallion y analítica en la nube con DuckDB

3.1. Descripción del problema

El patrón Medallion organiza un Data Lake en zonas sucesivas, cada una de las cuales conserva los datos en un nivel distinto de refinamiento y atiende a un consumidor distinto. El sentido de la separación no es el orden: cada zona responde una pregunta diferente y por lo tanto está sujeta a un contrato diferente.

La zona cruda responde “¿qué nos entregó realmente la fuente?” y su contrato es la fidelidad —almacena los datos tal como se recibieron, de modo que cualquier transformación posterior pueda auditarse o repetirse—. La zona optimizada responde “¿qué datos podemos considerar confiables y consultar con eficiencia?” y su contrato es la validez más una disposición física eficiente. La zona de consumo responde “¿qué quiere mirar el negocio?” y su contrato es que contenga exactamente las métricas que un tablero necesita, ya calculadas.

La tarea de este punto fue materializar la segunda y la tercera zona en S3, y construir la capa de

consumo con un motor analítico de consultas en lugar de una librería de dataframes de propósito general.

3.2. Método de solución

La Tabla 4 especifica las tres zonas tal como quedaron implementadas en el bucket `s3://juanpablojimenez`.

Zona	Clave del objeto	Formato	Contrato
Cruda	<code>raw_zone/ restaurants_raw.csv</code>	CSV	Fidelidad a la fuente; la traza de auditoría del pipeline.
Optimizada	<code>optimized_zone/ restaurants_processed .parquet</code>	Parquet (Snappy)	Registros validados en disposición columnar; la tabla de detalle analítico.
Consumo	<code>consumption_zone/ category_kpis.parquet</code>	Parquet (Snappy)	KPI preagregados por categoría; la superficie que lee una herramienta de BI.

Tabla 4: Especificación de las tres zonas del Data Lake.

Fuente: elaboración propia.

Una decisión deliberada de diseño. Tras cargar el CSV a la zona cruda, el pipeline no continúa trabajando con el DataFrame que ya tiene en memoria: lee el objeto de vuelta desde S3 con `get_object()` y lo vuelve a parsear. Esto cuesta un viaje de red adicional, y lo adoptamos a propósito. Convierte a la zona cruda en la fuente de verdad efectiva de todo lo que viene después, en lugar de ser un efecto secundario que se escribe de paso; y significa que una corrupción introducida durante la carga, o una discrepancia entre lo que creemos haber escrito y lo que S3 efectivamente almacenó, se detecta de inmediato y no más adelante, cuando las dos copias ya divergieron.

La capa de consumo. La agregación se expresa como una consulta SQL que DuckDB ejecuta contra el archivo Parquet local, y su resultado se escribe como un segundo archivo Parquet:

Algoritmo 3. Consulta de agregación de la capa de consumo (DuckDB)

```
SELECT
  category_name          AS category,
  COUNT(*)               AS total_restaurants,
  ROUND(AVG(distance), 2) AS avg_distance_m,
  MIN(distance)          AS min_distance_m,
  MAX(distance)          AS max_distance_m,
  ROUND(AVG(latitude), 6) AS centroid_lat,
  ROUND(AVG(longitude), 6) AS centroid_lon
FROM read_parquet('output/restaurants_processed.parquet')
GROUP BY category_name
ORDER BY total_restaurants DESC, avg_distance_m ASC
```

El mismo resultado podría obtenerse en Pandas con una sola llamada a `groupby().agg()`, y al volumen de datos que manejamos esa alternativa correría por lo menos igual de rápido. Escogimos DuckDB por tres razones que son arquitectónicas y no de desempeño a esta escala. Primera, la consulta es portable: reemplazar `read_parquet()` por una tabla registrada en un catálogo produce, casi carácter por carácter, la consulta que se ejecutaría en Amazon Athena contra la misma zona optimizada, mientras que una expresión de Pandas no se transfiere a ningún lado. Segunda, DuckDB lee el pie del archivo Parquet y materializa únicamente las columnas que la consulta proyecta—cuatro de las seis disponibles en nuestro caso—, que es el mismo mecanismo de poda de columnas que determina el volumen facturable de Athena, de modo que la ejecución local ensaya el modelo de costos de la nube. Tercera, DuckDB ejecuta fuera de memoria principal, sobre conjuntos más grandes que la RAM disponible, cosa que Pandas no hace (Raasveldt y Mühleisen, 2019).

3.3. Resultados

La consulta produjo 26 grupos de categoría a partir de los 50 registros validados. La Tabla 5 reproduce los ocho grupos que contienen más de un establecimiento; las dieciocho categorías restantes contienen un único local cada una.

Categoría	Locales	Prom. (m)	Mín. (m)	Máx. (m)	Centroide lat.
Restaurant	8	500,25	336	874	4,861039
BBQ Joint	7	784,43	363	1352	4,861419
Burger Joint	4	1101,75	292	1519	4,866288
Colombian Restaurant	3	291,67	246	321	4,861237
Pizzeria	3	361,33	183	473	4,861272
Italian Restaurant	3	674,33	430	1095	4,862102
Sushi Restaurant	2	1068,50	954	1183	4,857559
Mexican Restaurant	2	1226,50	1175	1278	4,866352

Tabla 5: Extracto de la capa de consumo: categorías con más de un establecimiento, ordenadas por número de locales. La tabla completa contiene 26 filas.

Fuente: elaboración propia.

Del agregado se desprenden dos observaciones. La distribución de categorías está fuertemente sesgada: los tres grupos mayores concentran diecinueve de los cincuenta establecimientos, mientras que dieciocho categorías están representadas por un solo local. Y las distancias promedio son informativas por sí mismas: *Colombian Restaurant* y *Pizzeria* se concentran entre unos 300 y 360 metros del punto de referencia, consistente con un centro tradicional de municipio, mientras que *Burger Joint* promedia más de un kilómetro, lo cual sugiere que esos establecimientos se agrupan sobre los corredores comerciales periféricos y no en el casco histórico.

Ambos objetos Parquet se cargaron correctamente, completando las tres zonas de la arquitectura.

4. Verificación programática y evidencia FinOps

4.1. Descripción del problema

Un pipeline que reporta su propio éxito no constituye evidencia de que el Data Lake esté en el estado esperado. Las llamadas de carga retornaron sin lanzar excepciones, pero eso solo prueba que las peticiones fueron aceptadas; no prueba que existan tres objetos, en tres prefijos distintos, con tamaños plausibles. La verificación debe hacerse interrogando a S3 y no confiando en el registro.

El enunciado pide además que esa verificación demuestre empíricamente el beneficio de compresión del formato columnar frente al CSV original. Como se documenta más adelante, nuestra medición no reprodujo esa expectativa al volumen de datos que maneja este taller, lo cual nos obligó a extender el análisis.

4.2. Método de solución

La tarea de verificación llama a `list_objects_v2()` de boto3 una vez por prefijo de zona, extrae los campos **Key** y **Size** de cada objeto devuelto y los ensambla en un DataFrame que se imprime a consola. Después localiza dentro de ese inventario el CSV crudo y el Parquet optimizado y calcula la diferencia relativa entre sus tamaños.

Habiendo obtenido un resultado contrario al esperado, añadimos una octava tarea al pipeline. Replica el conjunto de datos validado con factores de 1, 10, 100 y 1000 —produciendo 50, 500, 5000 y 50 000 registros—, serializa cada réplica a CSV y a Parquet en archivos temporales locales, y registra los tamaños resultantes. Los archivos temporales se eliminan de inmediato y no se escribe ningún objeto en S3, de modo que el experimento no incurre en costo de almacenamiento ni consume presupuesto del Learner Lab.

4.3. Resultados

La Tabla 6 reproduce el inventario devuelto por `list_objects_v2()`.

Zona	Clave del objeto	Bytes	KB
raw_zone	raw_zone/restaurants_raw.csv	4.220	4,12
optimized_zone	optimized_zone/restaurants_processed.parquet	6.955	6,79
consumption_zone	consumption_zone/category_kpis.parquet	6.143	6,00

Tabla 6: Inventario del Data Lake obtenido programáticamente con `boto3.list_objects_v2()`.

Fuente: elaboración propia.

La medición contradice la premisa del ejercicio. Con cincuenta registros el archivo Parquet ocupa 6.955 bytes frente a 4.220 del CSV: es un 64,8% más grande, no más pequeño. Esto no es un defecto del pipeline sino una propiedad del formato. Un archivo Parquet carga con una sobrecarga estructural fija que es independiente del número de filas: los bytes mágicos que identifican el formato,

los metadatos por fragmento de columna, las páginas de diccionario y un pie de archivo que almacena el esquema junto con estadísticas de cada grupo de filas (Apache Software Foundation, 2026). Esa sobrecarga suma unos pocos kilobytes. Un CSV, en cambio, no tiene prácticamente sobrecarga fija: una línea de encabezado y nada más. Por debajo de cierto volumen el costo fijo domina el archivo, y el CSV gana.

La Tabla 7 y la Figura 2 presentan el experimento de escalamiento, que localiza el volumen en el que la relación se invierte.

Registros	CSV (bytes)	Parquet (bytes)	Razón P/C	Ahorro
50	4.220	6.955	1,648	−64,8 %
500	41.696	7.359	0,176	82,4 %
5.000	416.456	8.580	0,021	97,9 %
50.000	4.164.056	21.955	0,005	99,5 %

Tabla 7: Serialización del mismo conjunto de datos replicado a volúmenes crecientes.

Fuente: elaboración propia.



Figura 2: Crecimiento del tamaño de archivo frente al número de registros para ambos formatos de serialización, en ejes logarítmicos. La línea vertical punteada marca el punto de equilibrio interpolado.

Fuente: elaboración propia a partir de la salida de la tarea de benchmark.

El comportamiento de las dos curvas es cualitativamente distinto, y esa diferencia es la sustancia del resultado. El CSV crece de forma lineal: multiplicar el número de registros por mil multiplica el tamaño del archivo por 987. El Parquet crece de forma sublineal, por un factor de apenas 3,2 en el mismo rango, porque la codificación por diccionario almacena cada valor repetido una sola vez y en

adelante lo referencia por índice, y porque la sobrecarga fija se amortiza sobre un número creciente de filas. Interpolando en escala logarítmica entre nuestras dos primeras mediciones, el punto de equilibrio se sitúa alrededor de los 85 registros; el primer volumen que medimos en el que el Parquet es inequívocamente menor es el de 500 registros, donde ya ahorra un 82,4 %. Con 50.000 registros el ahorro alcanza el 99,5 %.

La conclusión honesta para este taller es, por lo tanto, que con cincuenta registros el argumento a favor del formato columnar no es el tamaño en almacenamiento. La justificación a esta escala está en otro lugar —en el volumen escaneado por consulta— y se desarrolla en la sección siguiente.

5. Discusión

5.1. La separación en zonas como práctica FinOps

La separación de un Data Lake en zonas cruda, optimizada y de consumo suele justificarse en términos de gobierno de datos, pero su fundamento económico es al menos igual de sólido, y se vuelve explícito en cuanto se conecta un motor de consultas serverless al lake.

Amazon Athena cobra 5 USD por terabyte de datos escaneados, sin cargo por el cómputo en sí mismo (Amazon Web Services, 2026). El costo de una consulta es entonces una función directa de cuántos bytes debe leer el motor, y toda decisión arquitectónica que reduzca ese número reduce la factura en la misma proporción. Las zonas son precisamente una decisión de esa clase: permiten que cada consumidor lea de la capa que corresponde a su necesidad, en lugar de forzar a toda consulta a recorrer el historial completo de datos crudos.

La consecuencia es que la capa que una consulta lee determina su costo. Un tablero que solo necesita el número de establecimientos por categoría puede leer la zona de consumo, que en nuestro caso contiene 26 filas; si consultara la zona optimizada escanearía 50 registros y recalcularía la agregación en cada actualización, y si consultara la zona cruda escanearía además columnas que no usa, en un formato que no permite omitirlas. Las diferencias son insignificantes a nuestra escala y decisivas a escala productiva, donde un tablero que se actualiza cada quince minutos emite unas 2.800 consultas al mes contra los mismos datos.

5.2. Formatos columnares y volumen facturable

El papel del formato columnar es más específico que la compresión, y es justamente donde el argumento se sostiene incluso a volúmenes en los que, como medimos, el Parquet ocupa más espacio que el CSV.

En un formato orientado a filas los valores de un registro son contiguos en disco, de modo que leer un solo campo obliga a recorrer el archivo entero. En un formato columnar los valores de un campo son contiguos, de modo que el motor puede leer las columnas que la consulta proyecta y omitir el resto: el mecanismo conocido como poda de columnas, cuyo linaje va desde Dremel (Melnik et al., 2010) hasta los principios de diseño que Abadi et al. (2013) sistematizaron para los almacenes columnares. AWS documenta el efecto en sus propios ejemplos de precios: la conversión a un formato

columnar permitió a una consulta evitar la lectura de tres cuartas partes de un archivo y redujo el volumen escaneado a la cuarta parte del original (Amazon Web Services, 2026).

Nuestra consulta de agregación ilustra el mismo principio. Proyecta cuatro de las seis columnas de la zona optimizada; contra un archivo Parquet lee únicamente esas cuatro, mientras que contra un CSV del mismo contenido tendría que leer todos los bytes para llegar a ellas. Adicionalmente, las estadísticas por grupo de filas almacenadas en el pie del Parquet permiten a un motor descartar bloques completos sin descomprimirlos cuando un predicado excluye su rango de valores. Ninguno de los dos mecanismos está disponible en un formato de texto orientado a filas, a ningún volumen.

De aquí se sigue una conclusión matizada que nuestras mediciones respaldan: el argumento a favor de Parquet en la zona optimizada no es que siempre ocupe menos espacio —en nuestro caso no lo hizo— sino que es el único de los dos formatos que le permite al motor leer menos que el archivo completo.

5.3. Seguridad: Zero Trust y credenciales de vida corta

La guía del taller sobre acceso a la API enuncia la regla de forma explícita: la llave nunca debe escribirse dentro del código. Nuestra implementación carga `FOURSQUARE_API_KEY` y las tres variables de AWS desde un archivo `.env` mediante `python-dotenv`, y el archivo `.gitignore` del repositorio excluye ese archivo del control de versiones, de modo que ninguna credencial queda presente ni en el código fuente ni en el historial publicado.

Las credenciales del Learner Lab encarnan por sí mismas el mismo principio de una manera que vale la pena señalar. Las emite AWS STS, expiran al terminar la sesión de cuatro horas y deben renovarse desde el panel “AWS Details”. Durante el desarrollo esto fue una fricción recurrente: el pipeline falló con `ExpiredToken` en más de una ocasión y cada recuperación exigió copiar de nuevo tres valores. Esa fricción es la propiedad de seguridad, no un defecto del entorno. Una credencial IAM permanente que se filtra sigue siendo válida hasta que alguien lo advierta y la revoque; una credencial con vigencia de cuatro horas acota la ventana de exposición por construcción. La respuesta correcta de ingeniería no es buscar credenciales de mayor duración sino hacer legible el vencimiento, que es lo que hace nuestra capa de traducción de errores.

Dos controles adicionales, configurados al crear el bucket, completan la postura: el acceso público está bloqueado a nivel de bucket, y el cifrado en reposo por defecto está habilitado con una llave de AWS KMS, que se aplica automáticamente a todo objeto que el pipeline carga.

5.4. Limitaciones y trabajo futuro

Conviene dejar registro de cuatro limitaciones de la implementación actual, en orden decreciente de impacto sobre el valor analítico del resultado.

El conjunto de datos está limitado a cincuenta registros. El endpoint de búsqueda de Foursquare devuelve a lo sumo cincuenta resultados por petición, y no implementamos la paginación por cursor que la API ofrece a través del encabezado `link` de la respuesta (Foursquare, 2026). Esta sola limitación es responsable del resultado contraintuitivo de compresión de la sección 4.3, y

también restringe la capa de consumo, como se anota a continuación. Implementar la paginación, y adicionalmente consultar varios puntos de referencia a lo largo de la región de Sabana Centro, produciría un conjunto de datos de una escala en la que los beneficios de la arquitectura serían directamente observables en lugar de requerir extrapolación.

La capa de consumo apenas agrega. Reduce 50 registros a 26 filas, y 18 de esas filas corresponden a un solo establecimiento. Una capa de consumo cuya cardinalidad se aproxima a la de su fuente no justifica su costo, puesto que la consulta que ahorra es trivial. La falla está en la elección de la dimensión de agrupación: la taxonomía de categorías de Foursquare es mucho más granular de lo que soporta el volumen de nuestra muestra. Una dimensión de menor cardinalidad —bandas de distancia, o una agrupación que consolide las categorías infrecuentes en un rubro “Otros”— produciría una capa que comprimiera de verdad.

La zona optimizada no está particionada. Almacenar el archivo Parquet bajo un esquema de particionamiento estilo Hive, por ejemplo por categoría o por fecha de ingesta, permitiría a un motor descartar archivos completos antes de abrirlos, añadiendo un nivel de poda más grueso por encima del que el formato ya provee internamente.

La zona cruda aplanar una estructura anidada. Como se explicó en la sección 1.2, el objeto de categorías se aplanar en la ingesta porque el CSV no puede representar anidamiento, y solo se conserva la categoría principal. Una lectura más estricta del patrón Medallion almacenaría la respuesta de la API tal cual, como JSON, en la zona cruda, y ejecutaría el aplanamiento en la transición hacia la zona optimizada, preservando tanto la fidelidad de la capa cruda como las categorías secundarias que un establecimiento pueda tener.

5.5. Conclusiones

La consolidación de tres scripts de ejecución manual en un único flujo orquestado cambió la naturaleza del artefacto y no simplemente su empaque. El orden de ejecución ahora lo impone el código, cada etapa reporta un estado explícito, y los modos de falla con los que nos habíamos topado por accidente durante el desarrollo se manejan ahora por diseño, con mensajes que enuncian la acción correctiva.

La capa de calidad de datos no descartó ningún registro en esta ejecución, y aun así la consideramos valiosa: su función es garantizar que una degradación futura de la fuente se detecte antes de propagarse, y verificamos contra un lote sintético que la ruta de aborto impide la publicación cuando el umbral no se cumple.

El resultado más instructivo del taller fue el que contradijo nuestra expectativa. Medir en lugar de suponer el beneficio de compresión del Parquet mostró que con cincuenta registros el formato columnar es un 64,8 % más grande que el CSV, y que la ventaja que la literatura le atribuye aparece solo por encima de unos cien registros, alcanzando el 99,5 % con cincuenta mil. El ejercicio dejó claro que el argumento FinOps a favor del formato columnar a pequeña escala no descansa en el tamaño de almacenamiento —el almacenamiento es, en todo caso, el componente más barato de una arquitectura analítica— sino en el volumen escaneado por consulta, que es lo que los motores serverless efectivamente facturan.

Referencias

Abadi, D., Boncz, P., Harizopoulos, S., Idreos, S., y Madden, S. (2013). The design and implementation of modern column-oriented database systems. *Foundations and Trends in Databases*, 5(3), 197–280.

Amazon Web Services. (2026). *Amazon Athena pricing*. <https://aws.amazon.com/athena/pricing/>

Apache Software Foundation. (2026). *Apache Parquet documentation: File format*. <https://parquet.apache.org/docs/file-format/>

FinOps Foundation. (2026). *FinOps framework overview*. <https://www.finops.org/framework/>

Foursquare. (2026). *Place search — Places API reference*. <https://docs.foursquare.com/fsq-developers-place-reference/place-search>

Melnik, S., Gubarev, A., Long, J. J., Romer, G., Shivakumar, S., Tolton, M., y Vassilakis, T. (2010). Dremel: Interactive analysis of web-scale datasets. *Proceedings of the VLDB Endowment*, 3(1–2), 330–339.

Prefect Technologies. (2026). *Prefect 3 documentation: Tasks and flows*. <https://docs.prefect.io/>

Raasveldt, M., y Mühleisen, H. (2019). DuckDB: An embeddable analytical database. *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*, 1981–1984.